Python development toolchain tutorial

Lukas Prader

prader@student.chalmers.se

Aim of this session

- Create a GitHub account
- Set up VSCode
- Connect Github and create a course repository
- Set up code formatting and project management with uv and Ruff
- Integrate openrouter into Github Copilot in VSCode
- Show how to use the full toolchain to solve the boolean function task

Follow-along tutorial with some detailed explanations

GitHub

- Go to https://github.com/ and create an account if you do not have one already

- Use a **personal email**
  you want this account to stay with you even after you finish university, you can add other email addresses later

- We will go through the tutorial at https://github.com/luprader/python_dev_toolchain

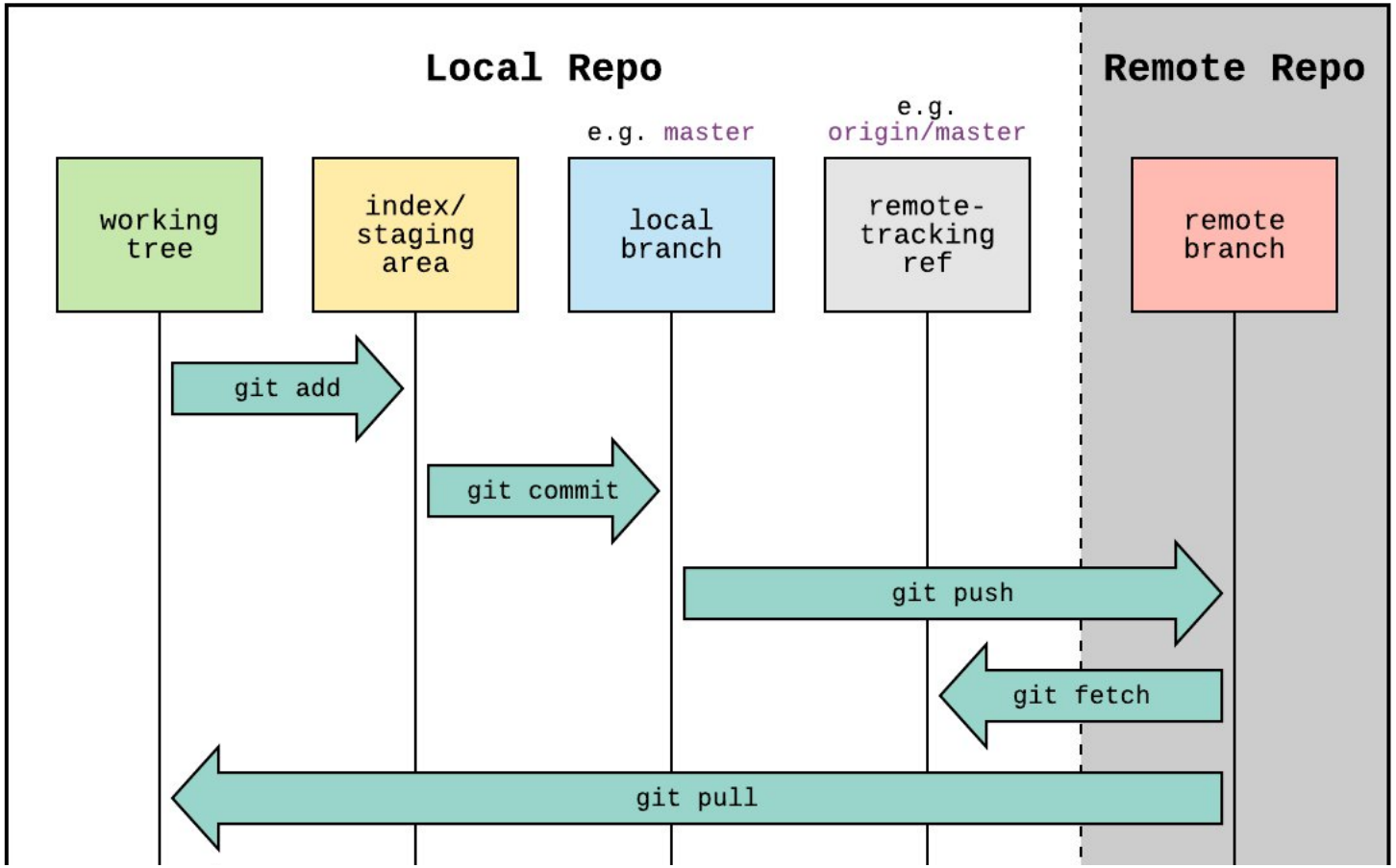
VSCode

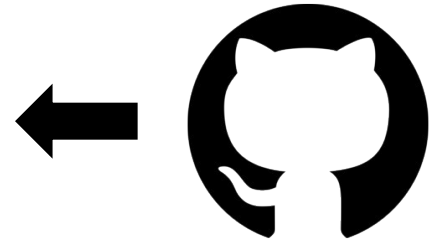
- It is currently the most popular code editor
- Multi-language support with extensions (even LaTeX)
- Many features to make coding easier

Git: saving future you from past you / others

- Store and share code
- Keep track of changes and make it easy to go back to old versions
- Allow collaboration between people

Git: saving future you from past you / others





GitHub

modified from https://imgur.com/oodiCnB

Git: saving future you from past you / others

- **git commit** creates a local checkpoint of staged files
- **git push** sends committed changes to GitHub
- **git pull** synchronises your local repository with any missing changes from the GitHub

Dependency management

- If packages change, your code might stop working
- Execution environment can influence code behaviour, Windows ≠ iOS ≠ Linux
- We want to make sure our results are **reproducible**
- Need a way to ensure consistency for the python version and library versions that we use in a project
- **uv:** modern and very popular package manager for Python

Formatting and linting

- Code should be readable by other people and follow best practices
- PEP8 are the code style guidelines for Python

- **Formatter**: Solves style inconsistencies in code without changing semantics
- **Linter**: Suggests code changes according to best practices

- **Ruff**, because it is fast and integrates with uv

Best practice in this course

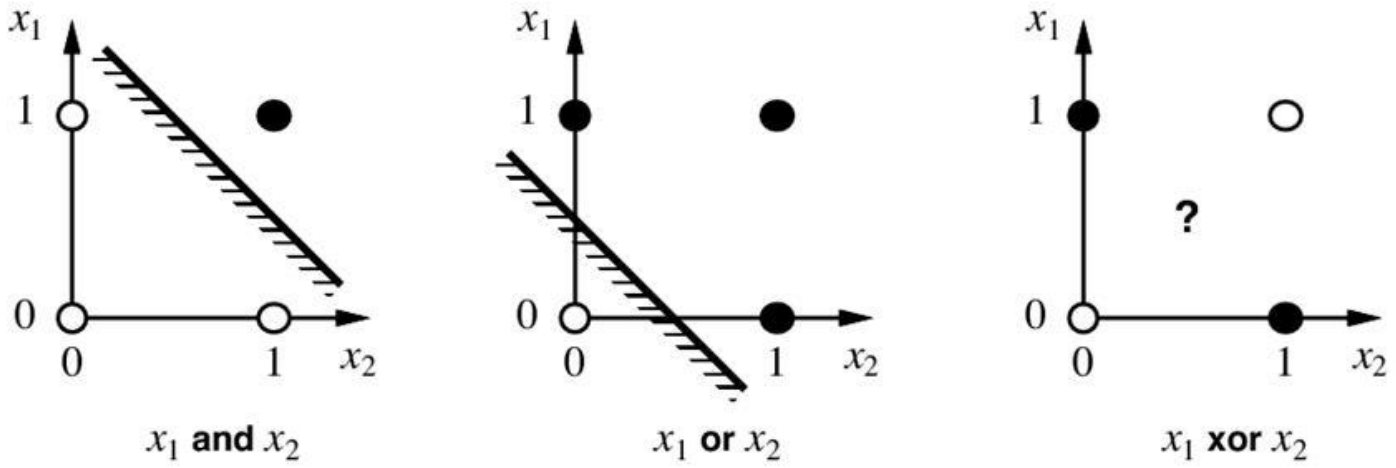
- If possible, write equations in their vector/matrix forms

$$\mathrm{b} = \sum_{j=1}^{n} w_j x_j - \theta \quad \rightarrow \quad \mathrm{b} = \vec{w} \cdot \vec{x} - \theta$$

- `numpy.array` vector operations are much more efficient than `for` loops over lists

HW 1 - Boolean functions

- An n-dimensional boolean function has binary input values and returns a binary output value. They have a nice geometric representation, relating to linear separability:

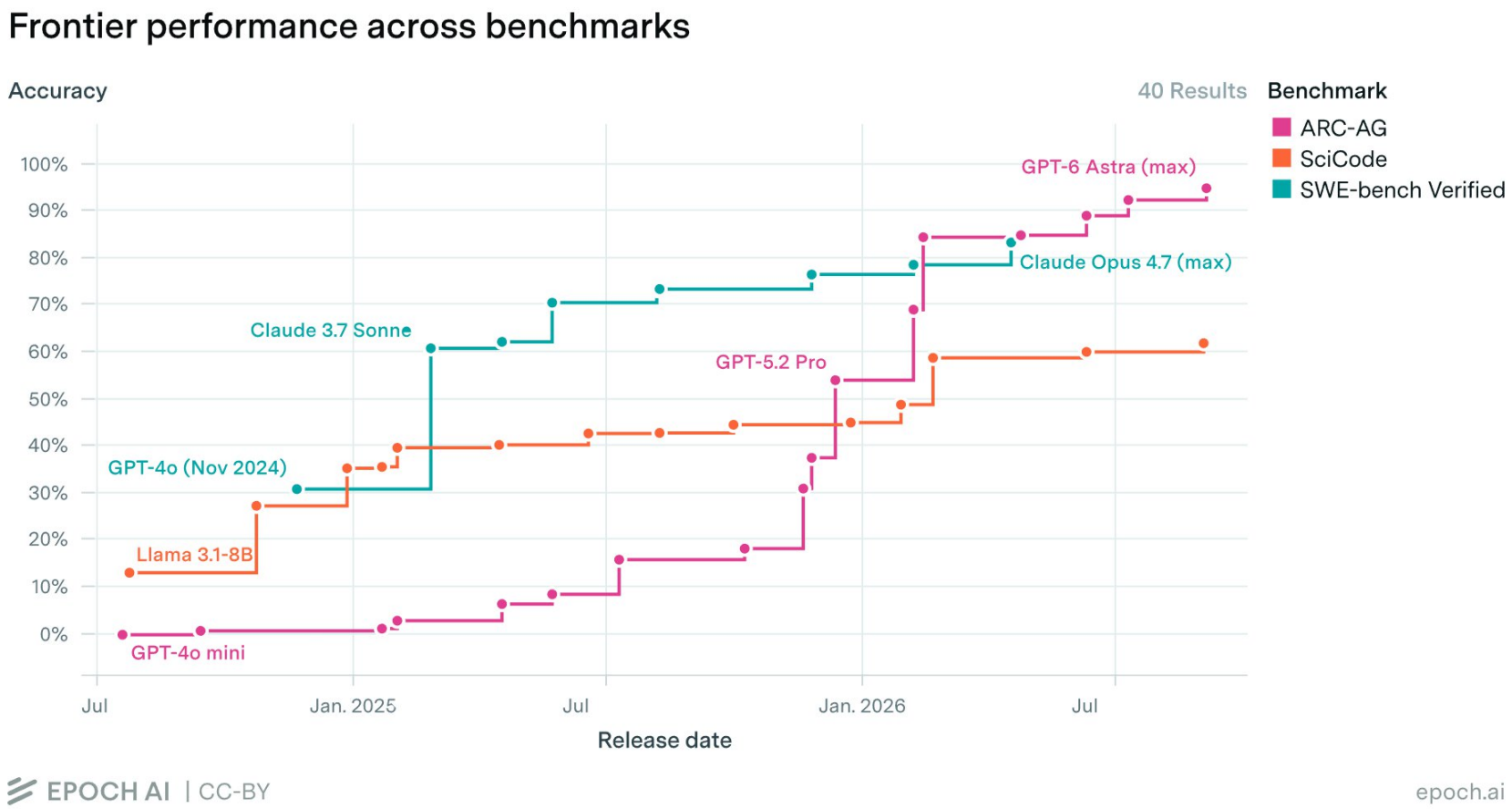


https://seeyong.github.io/2018/05/neural-net-for-xor-1/

But our task uses McCulloch-Pitts neurons, so our inputs are ±1, not like in the image

Using LLMs as Coding tools

Can you write better code than frontier LLMs?



https://epoch.ai/benchmarks?view=graph&tab=benchmarks&benchmark-selection=%7ESWE-Bench+verified%7Earc_agi_2%7Escicode

Can you write better code than frontier LLMs?

SciCode:

- 80 formulated problems across STEM fields with "gold-standard" solutions, divided into sub-problems (338 total)
- The problems require skills in numerical methods, simulation of systems and scientific calculation as well as scientific reasoning and knowledge about the research field

- Performance evaluated with numerical and domain specific test-cases

Can you write better code than frontier LLMs?

SWE-bench Verified:

- Human verified (to be solvable etc.) subset of SWE-bench

- SWE-bench tests AI systems' ability to solve GitHub issues.
- 2,294 task instances collected from pull requests and issues from 12 popular Python repositories which have existing testing suites.

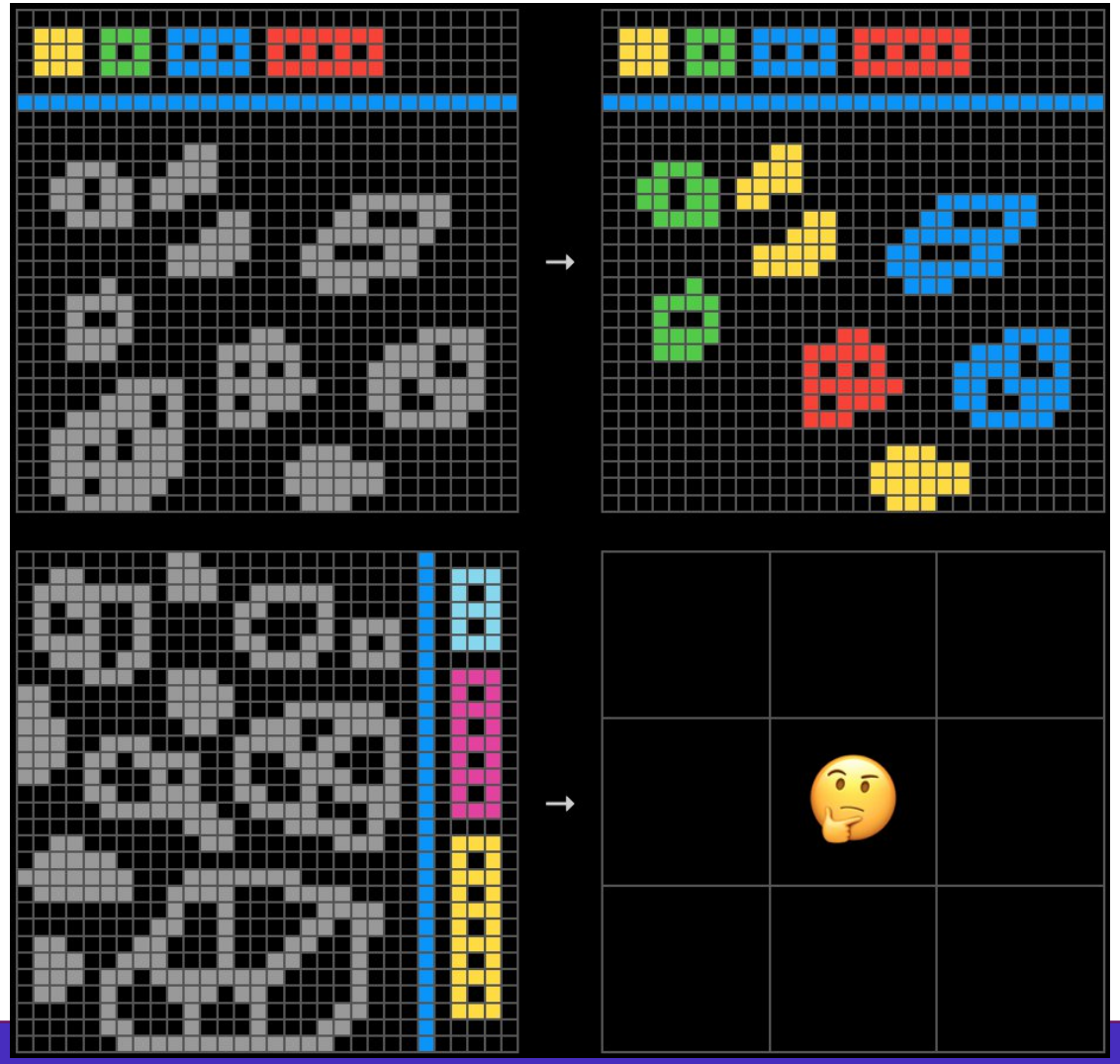
- LLM code has to resolve the issue and pass the pre-existing test cases of the issue.

Can you write better code than frontier LLMs?

- ARC-AGI-2:

- Symbolic Interpretation

Tasks requiring symbols to be interpreted as having meaning beyond their visual patterns.



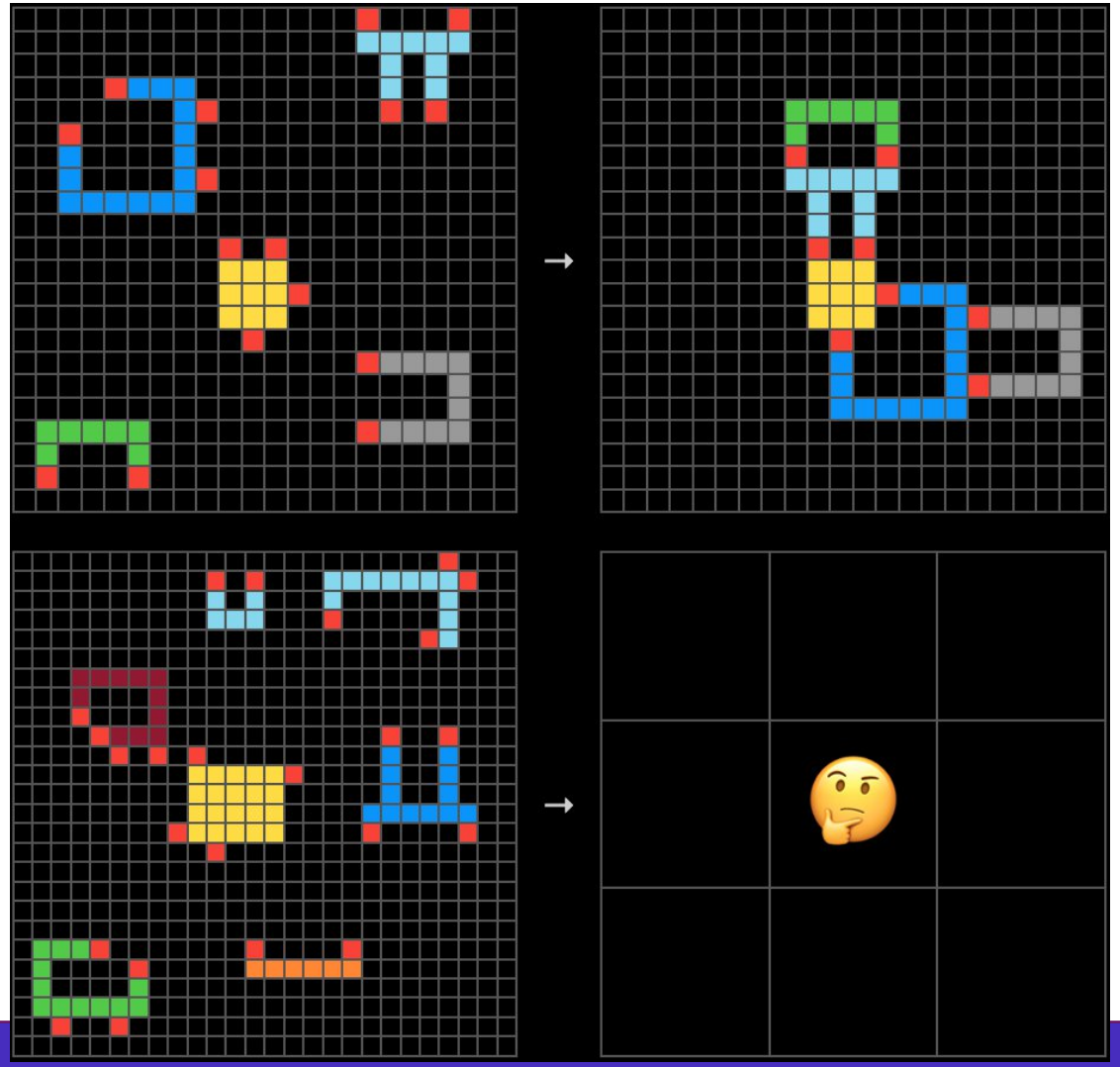
https://arcprize.org/arc-agi/2

Can you write better code than frontier LLMs?

- ARC-AGI-2:

- Compositional Reasoning

Tasks requiring simultaneous application of a rules, or application of multiples rules that interact with each other.



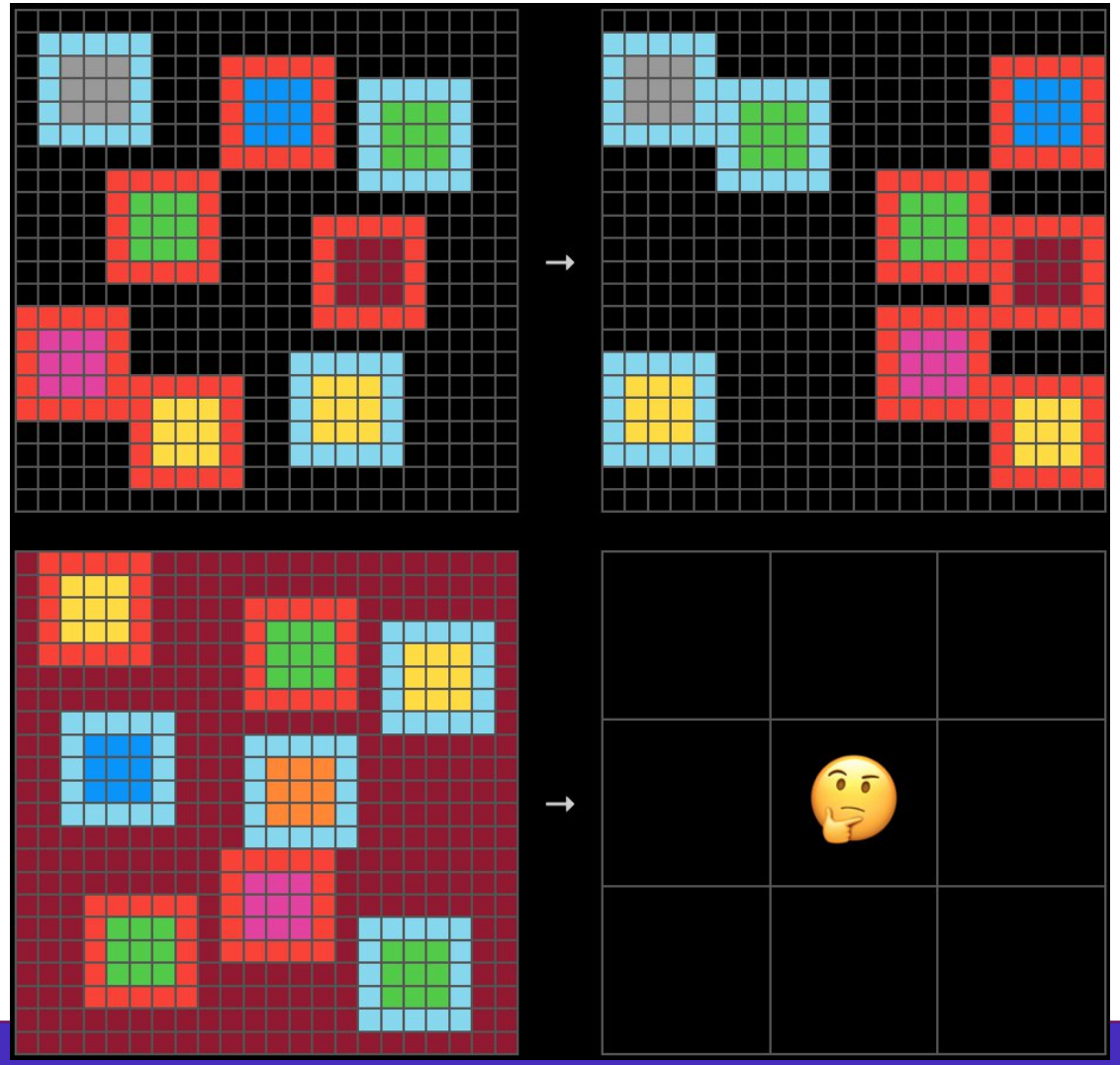
https://arcprize.org/arc-agi/2

Can you write better code than frontier LLMs?

- ARC-AGI-2:

- Contextual Rule Application

Tasks where rules must be applied differently based on context.



https://arcprize.org/arc-agi/2

Can you write better code than frontier LLMs?

- ARC-AGI-2:

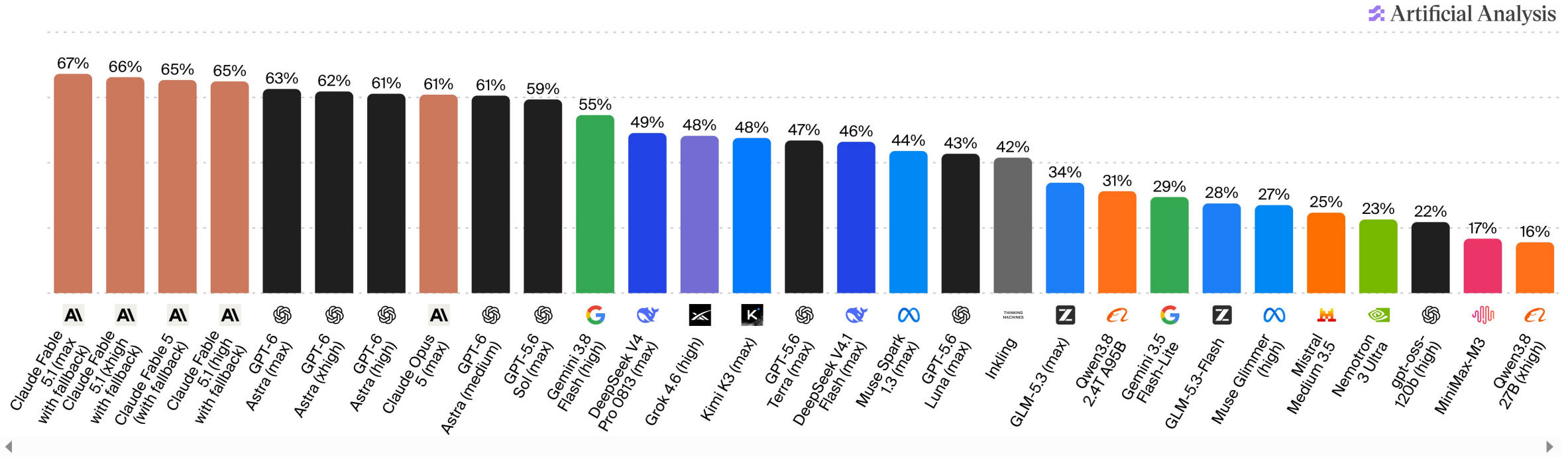
- Human contestants on average solve 66%
- ARC-AGI-2 was "beaten" when models achieved 85%

- There is now also ARC-AGI-3 for agentic intelligence, with GPT-6 Astra recently achieving > 95%

https://arcprize.org/arc-agi/2

How accurate are LLMs in general?

Factual recall and knowledge calibration across 6,000 questions covering:
Business, Humanities & Social Sciences, Science, Engineering & Mathematics, Health, Law and Software Engineering



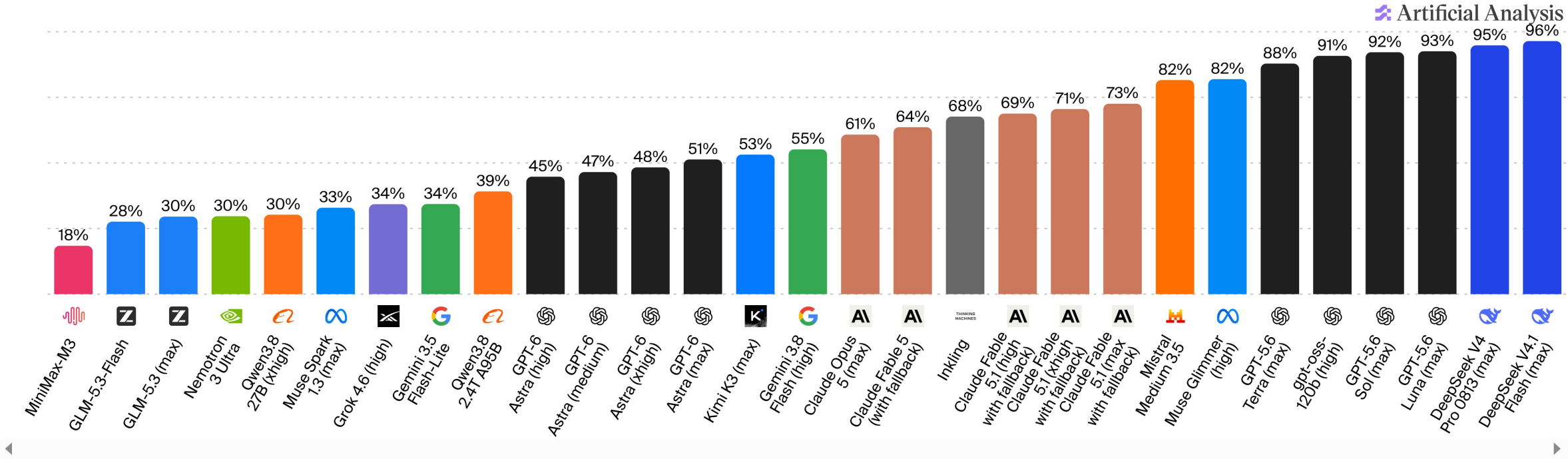
https://artificialanalysis.ai/evaluations/omniscience

How accurate are LLMs in general?

AA-Omniscience Hallucination Rate

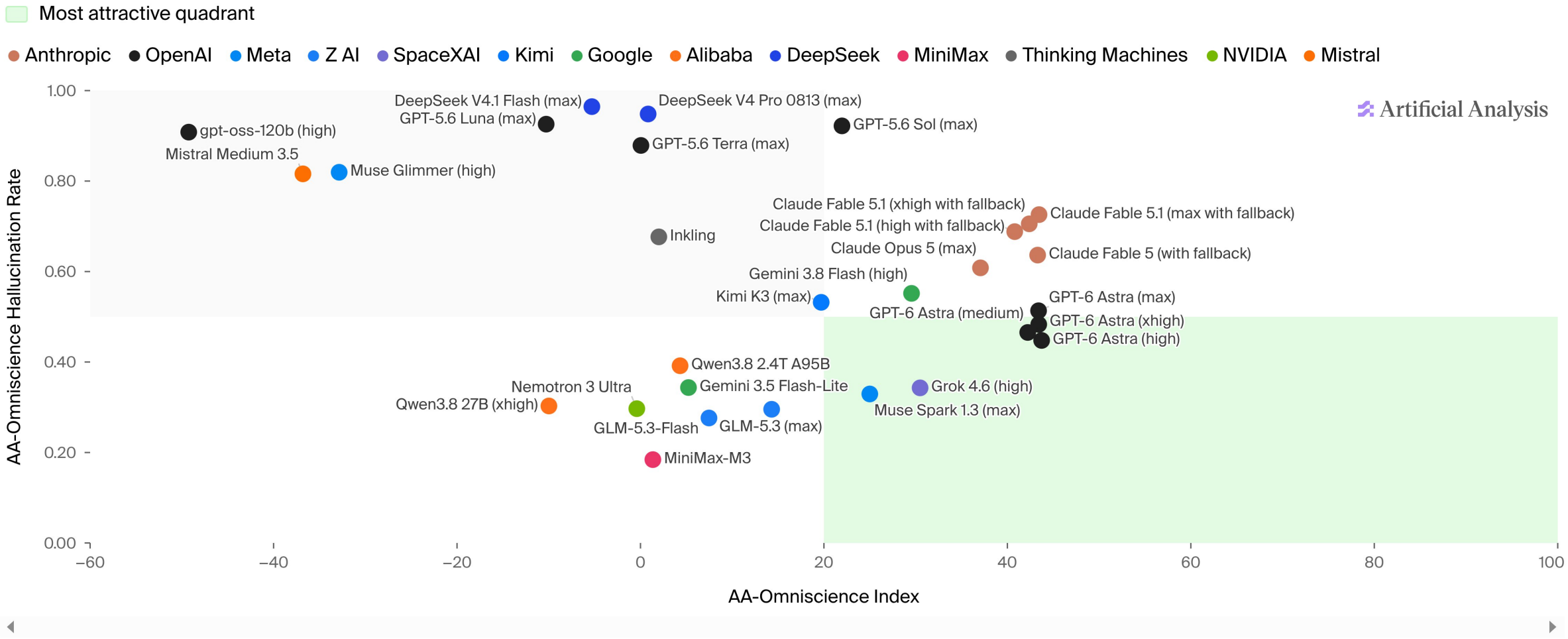
AA-Omniscience Hallucination Rate (lower is better) measures how often the model answers incorrectly when it should have refused or admitted to not knowing the answer. It is defined as the proportion of incorrect answers out of all non-correct responses, i.e. incorrect / (incorrect + partial answers + not attempted)

Factual recall and knowledge calibration across 6,000 questions covering:
Business, Humanities & Social Sciences, Science, Engineering & Mathematics, Health, Law and Software Engineering



https://artificialanalysis.ai/evaluations/omniscience

How accurate are LLMs in general?



https://artificialanalysis.ai/evaluations/omniscience

Know what AI regulations apply to you

- Chalmers regulations for AI use in thesis work
- GU regulations for AI use in studies
- Chalmers Library AI guidelines

Think about legal/copyright/ethical implications